SISTEMA DE GESTIÓN DE RESERVAS HOTELERAS



UNIVERSIDAD DON BOSCO

Integrantes:	**N° Carnet:**	**Rol:**
Susana Nicole Valle Méndez	VM253306	Desarrollador Backend Y frontend
Nayeli Eunice Huezo Guevara	HG251807	Analista de diseño de base datos
Ricardo Daniel Alfaro Tomasino	AT232988	Coordinador

Fecha de entrega: Domingo 29 de Octubre de 2026

Docente: Ing. Mario Alvarado

Asignatura: Desarrollo de Aplicaciones con Web Frameworks

Índice

3. Descripción del proyecto 3
3.1 Objetivo general 3
3.2 Objetivos específicos 3
4. Gestión integral del proyecto 3
4.1 Gestión de la integración 3
4.2 Gestión del alcance 4
4.3 Gestión del cronograma 5
4.4 Gestión de costos 6
4.5 Gestión de la calidad 7
4.6 Gestión de los recursos 9
4.7 Comunicación en equipo 9
4.8 Gestión de adquisición y cierre 9
5. Descripción técnica 11
5.1 Prototipo de la parte FrontEnd 11
5.2 Gestión de la base de datos 6
6. Resultados y conclusiones 4
7. Anexos y documentación de apoyo 4

3. Descripción del proyecto

3.1 Objetivo general

Desarrollar un sistema backend mediante una API REST en Java con Spring Boot que permita gestionar de forma integral los procesos de administración y reserva de habitaciones de una cadena hotelera, aplicando principios de arquitectura orientada a servicios (SOA) que garanticen consistencia de datos, escalabilidad y una experiencia de reserva completa para el usuario final, desde la búsqueda de disponibilidad hasta la confirmación de pago.

3.2 Objetivos específicos

- Diseñar una base de datos relacional que modele de forma consistente hoteles, habitaciones, huéspedes, reservaciones, pagos y servicios adicionales.
- Implementar un módulo de búsqueda que permita filtrar habitaciones disponibles según ubicación, fechas y número de huéspedes.
- Desarrollar el flujo completo de reserva, desde el registro de datos del huésped hasta la confirmación del pago.
- Establecer una gestión de usuarios y roles que controle el acceso a los distintos procesos del sistema.
- Generar estadísticas básicas de ocupación, reservaciones y cancelaciones que apoyen la toma de decisiones del hotel.

4. Gestión integral del proyecto

4.1 Gestión de la integración

El equipo gestionará los recursos y herramientas del proyecto mediante GitHub, donde se centralizará el control de versiones del código fuente, utilizando ramas por funcionalidad y revisiones (pull requests) antes de fusionar cambios a la rama principal. El seguimiento de tareas y avances se realizará en Trello, organizando el trabajo en tableros por estado (pendiente, en progreso, completado).

Se empleará la metodología ágil Scrum, trabajando en sprints con entregas parciales que permitan revisar el avance del proyecto de forma incremental.

4.2 Gestión del alcance

El alcance del proyecto comprende el desarrollo del backend del sistema de reservas hoteleras mediante una API REST, incluyendo las siguientes funcionalidades mínimas:

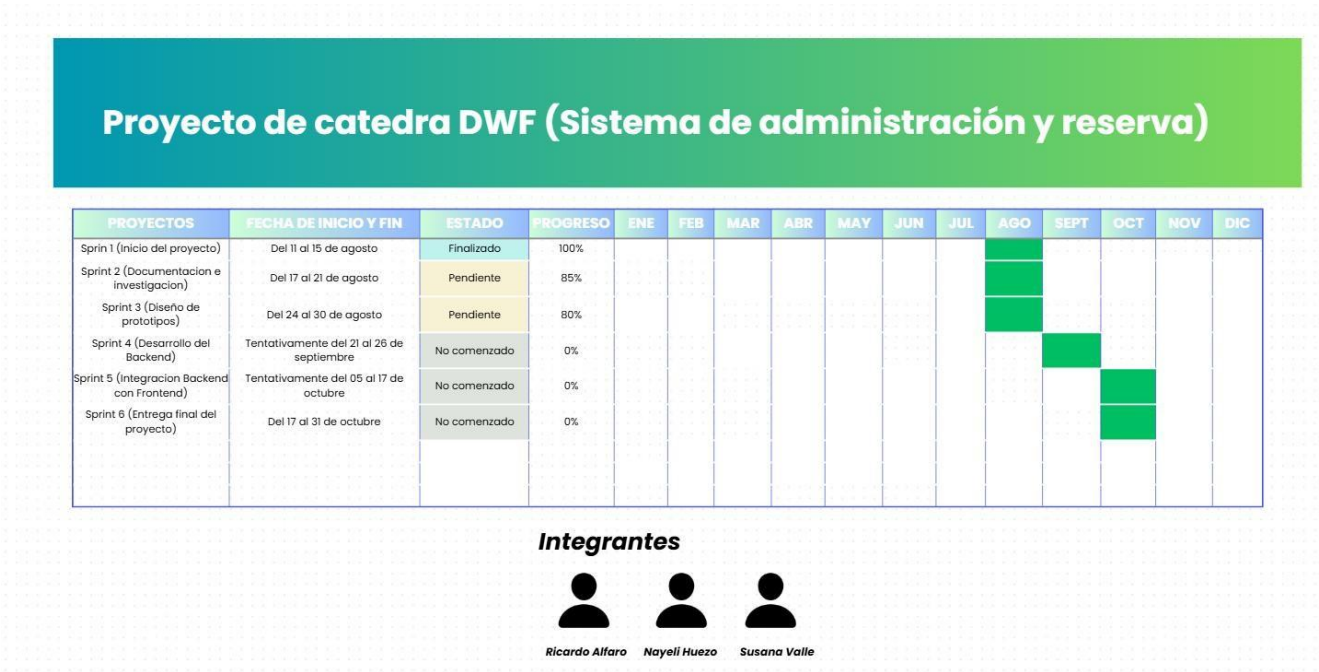
- Registro y gestión de hoteles, sucursales, habitaciones, tipos de habitación y tarifas.
- Búsqueda y visualización de disponibilidad según criterios de ubicación, fechas y números de huéspedes.
- Registro de huéspedes y gestión completa del proceso de reserva (selección, verificación, confirmación y pago).
- Gestión de cancelaciones y modificaciones de reservas según políticas del hotel.
- Administración de servicios adicionales (alimentación, transporte, lavandería).
- Procesamiento de reclamos y solicitudes de huéspedes.
- Generación de estadísticas básicas de ocupación, reservaciones, cancelaciones e ingresos.
- Gestión de usuarios y roles con distintos niveles de acceso al sistema.

Quedan fuera del alcance de este proyecto:

La implementación de pasarelas de pago reales (se simulará el registro del pago), la app móvil nativa, y la integración con sistemas de terceros (motores de reserva externos, redes de distribución hotelera).

Cualquier cambio en el alcance, integrantes del equipo o funcionalidades definidas deberá ser acordado por todo el equipo y documentado, ya que modificaciones no autorizadas afectan directamente la evaluación del proyecto.

4.3 Gestión del cronograma



4.4 Gestión de costos

Presupuesto Sistema de administración y reservas de hotel			
Herramientas y recursos	Cantidad a utilizar	Licencias	Costo del recurso
GitHub	Únicamente un repositorio	Gratuito	0
Gestor de bases de datos MySQL	3 (Una instalación para cada integrante)	Gratuito	0
IntelliJ IDEA	3 (Una instalación para cada integrante)	Versión gratuita	0
Draw.io	1	Gratuito	0
Spring Boot	3 (Una instalación para cada integrante)	Gratuito	0
Trello	1	Versión gratuita	0
Postman	1	Gratuito	0
Vue, Vite y JavaScript	3 (Una instalación para cada integrante)	Gratuito	0
Presupuesto para infraestructura			
Servicios	Periodo de contratación	Plan	Costo
Hosting y dominio (Hostinger)	Anual	Premium	$35.88
Instancia en AWS	Mensual	Lightsail Linux	$7.00
Base de datos en AWS	Mensual	Lightsail Managed Database	$15.00
Internet	Dependiendo del proveedor	Dependiendo del proveedor	

Presupuesto de Recurso Humano			
Equipo de desarrollo	Periodo de contratación	Horas	Costo
Desarrollador del lado Backend y Frontend	Por proyecto	8	$640.00
Analista de base de datos	Por proyecto	8	$760.00
Coordinador de proyecto	Por Proyecto	8	$800.00

4.5 Gestión de la calidad

Para este punto se tendrá como objetivo revisar el desarrollo del sistema de reservas de hotel mediante pruebas, las cuales permitirán encontrar errores (bugs) y posibles mejoras que se puedan ir aplicando al proyecto a medida que avanza, antes de entregar la versión final. Asimismo, podrán surgir nuevas ideas o métodos que se puedan aplicar y, sobre todo, se realizarán validaciones para evitar que el proyecto tenga vulnerabilidades de seguridad graves.

Como equipo se ha decidido utilizar un flujo de trabajo basado en Git y GitHub, lo cual permitirá un mejor manejo de las versiones del proyecto y, en caso de accidentes, el repositorio funcionará como un "Backup". De igual manera, cada integrante irá subiendo al repositorio el desarrollo que realice, para ir juntando cada aporte al final.

Para ser un poco más específicos, este sería el flujo propuesto a implementar:

Estructura de ramas

Main: nuestra rama principal, a la cual se irán uniendo los avances. Se espera que contenga al final la versión que se pondrá en producción.

Una rama de Develop para cada integrante del equipo: para tener un mejor control de versiones y evitar errores al subir los cambios, cada integrante tendrá su propia rama de desarrollo, de modo que un accidente involuntario no afecte los avances de los demás.

Test: una rama dedicada únicamente a desarrollar las pruebas de calidad antes de implementarlas en la rama Main, ya que esta contendrá la versión final del proyecto.

Separación de Backend y Frontend

Para este punto es importante mantener un orden y una separación adecuada entre el Backend y el Frontend, ya que cada una de estas partes tendrá responsabilidades diferentes dentro del sistema. El punto de partida será el desarrollo del Backend, debido a que este será el encargado de procesar la información, aplicar las reglas de negocio, comunicarse con la base de datos y proporcionar los datos que posteriormente serán utilizados por el Frontend. Aunque ambas partes utilizarán tecnologías diferentes, estarán conectadas mediante una API REST, permitiendo que el Frontend pueda solicitar información al Backend y mostrarla al usuario de una forma más amigable.

Del lado del Backend se utilizarán tecnologías como Java y Spring Boot. Para mantener una estructura organizada, será necesario seguir las convenciones y estándares de programación establecidos para Java. Las clases deberán contar con nombres descriptivos, fáciles de comprender y relacionados con la responsabilidad que desempeñan dentro del sistema. Se evitarán nombres demasiado largos, o que puedan generar confusión entre diferentes componentes. Para las clases se utilizará el formato PascalCase.

También se buscará mantener una separación dentro del Backend: los Controllers serán utilizados para recibir y responder las solicitudes realizadas a través de la API, los Services contendrán la lógica principal del negocio y los Repositories estarán encargados de la comunicación con la base de datos. De esta manera se evitará colocar toda la lógica dentro de una sola clase.

Por otro lado, el Frontend será desarrollado utilizando Vue, el cual permitirá organizar la interfaz mediante componentes reutilizables. Esta estructura permitirá dividir la aplicación en diferentes partes, por ejemplo, componentes para realizar la búsqueda de habitaciones, mostrar las habitaciones disponibles, registrar los datos del huésped, mostrar el resumen de una reserva y realizar otras operaciones relacionadas con el sistema. De esta manera, cada componente tendrá una función específica y podrá ser reutilizado cuando sea necesario. También se mantendrá una separación entre los componentes visuales y los servicios encargados de comunicarse con la API. Las solicitudes al Backend deberán realizarse de una manera clara y ordenada, evitando colocar directamente toda la lógica de comunicación dentro de los componentes de la interfaz.

Criterios de calidad

Los criterios que usaremos para manejar el control de calidad serán los siguientes:

- Funcionalidad del código
- Manejo de base de datos
- Pruebas de datos a la API REST
- Integración de backend y frontend
- Validación y errores
- Documentación
- Control de versiones en Git

4.6 Gestión de los recursos

Para este punto es necesario organizar mejor las responsabilidades de cada integrante del equipo, debido a que el proyecto contempla diferentes áreas, como el desarrollo del Backend, el Frontend, la documentación y la gestión. Será necesario asignar responsabilidades específicas para evitar que las tareas se concentren en una sola persona, facilitando así cada actividad asignada.

La asignación de roles se pensó de la siguiente manera:

Coordinador del proyecto

El coordinador será responsable de coordinar las actividades generales del equipo y dar seguimiento al cumplimiento de las tareas establecidas en el cronograma. También tendrá como función verificar que las diferentes actividades se estén realizando dentro de los tiempos establecidos y que los avances de cada integrante sean registrados correctamente.

Desarrollador Backend y Frontend

El proyecto se realizará en equipo: cada integrante hará un aporte dentro del código y subirá cambios al repositorio, pero habrá una persona que dará inicio al desarrollo del Backend y, una vez que como equipo terminemos esa parte y hayamos realizado pruebas, esta persona también dará inicio al desarrollo del Frontend. En otras palabras, esta persona será quien inicie la estructura de ambos entornos.

Analista de base de datos y Responsable de Documentación

Este integrante será el encargado de mantener organizada la documentación técnica del proyecto y de registrar la información relacionada con los endpoints desarrollados: los métodos HTTP, los parámetros y datos requeridos y las respuestas esperadas. También apoyará con la documentación de los avances del proyecto.

En cuanto a la base de datos, esta persona dará inicio a la lógica que tendrá la base, desarrollará los diagramas de entidad-relación para comprender mejor la lógica del proyecto y creará un diccionario de datos.

4.7 Comunicación en equipo

Para mantener la comunicación entre los integrantes del proyecto, el medio principal será WhatsApp, para lo cual se creó un grupo en dicha aplicación. De ser necesario, también se podrán planear reuniones virtuales por Microsoft Teams o Google Meet.

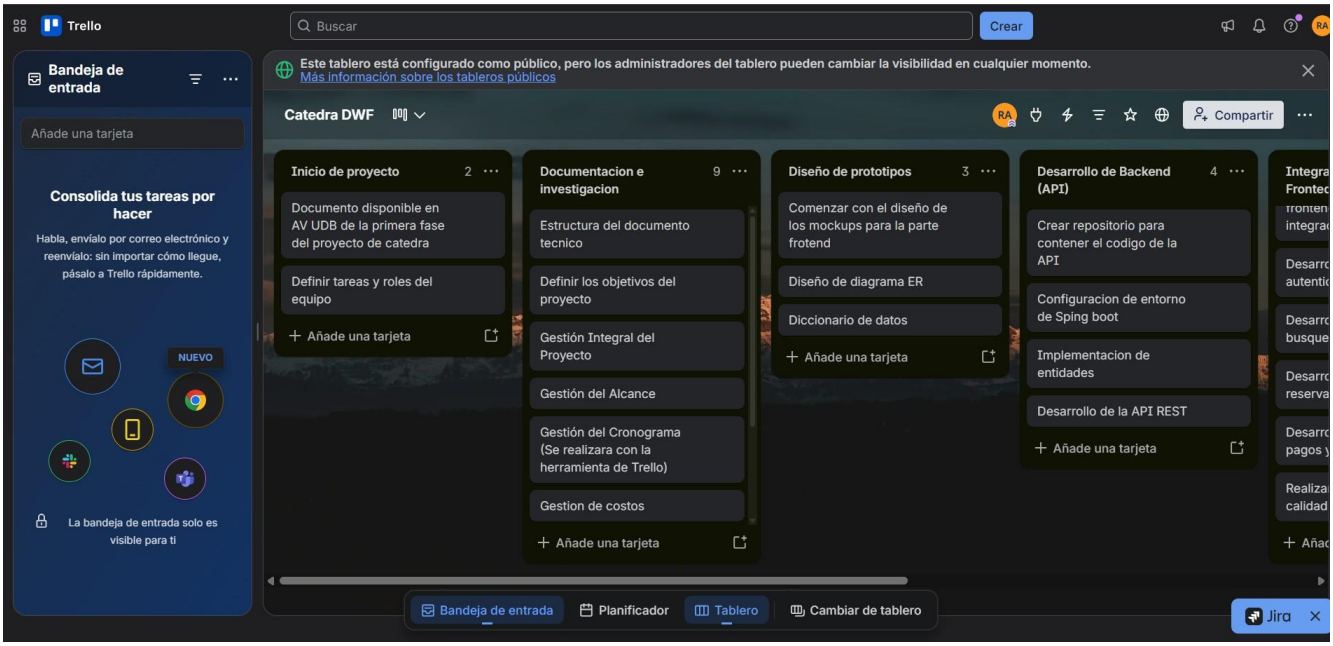
Además, se optó por utilizar un tablero en Trello para llevar una mejor organización de las actividades y, de ser necesario, que sirva como recordatorio de lo que hará cada integrante.

Por último, GitHub será utilizado como plataforma para el control de versiones y la colaboración sobre el código fuente. Cada integrante podrá trabajar sobre las funcionalidades que le hayan sido asignadas y posteriormente compartir sus cambios mediante el repositorio del proyecto.

4.8 Gestión de adquisición y cierre

Para este punto se tomarán en cuenta todos los elementos necesarios para realizar la entrega final de manera organizada. Como parte de esta etapa se contará con un repositorio donde se almacenarán el código fuente y la documentación relacionada con el proyecto. Además, como herramienta de gestión se decidió utilizar Trello, que permitirá dar seguimiento a las actividades realizadas durante las diferentes fases, mantener un registro del trabajo realizado y facilitar la comprobación de las tareas completadas.

Ejemplo de cómo se distribuyó cada actividad



Enlace del tablero de Trello

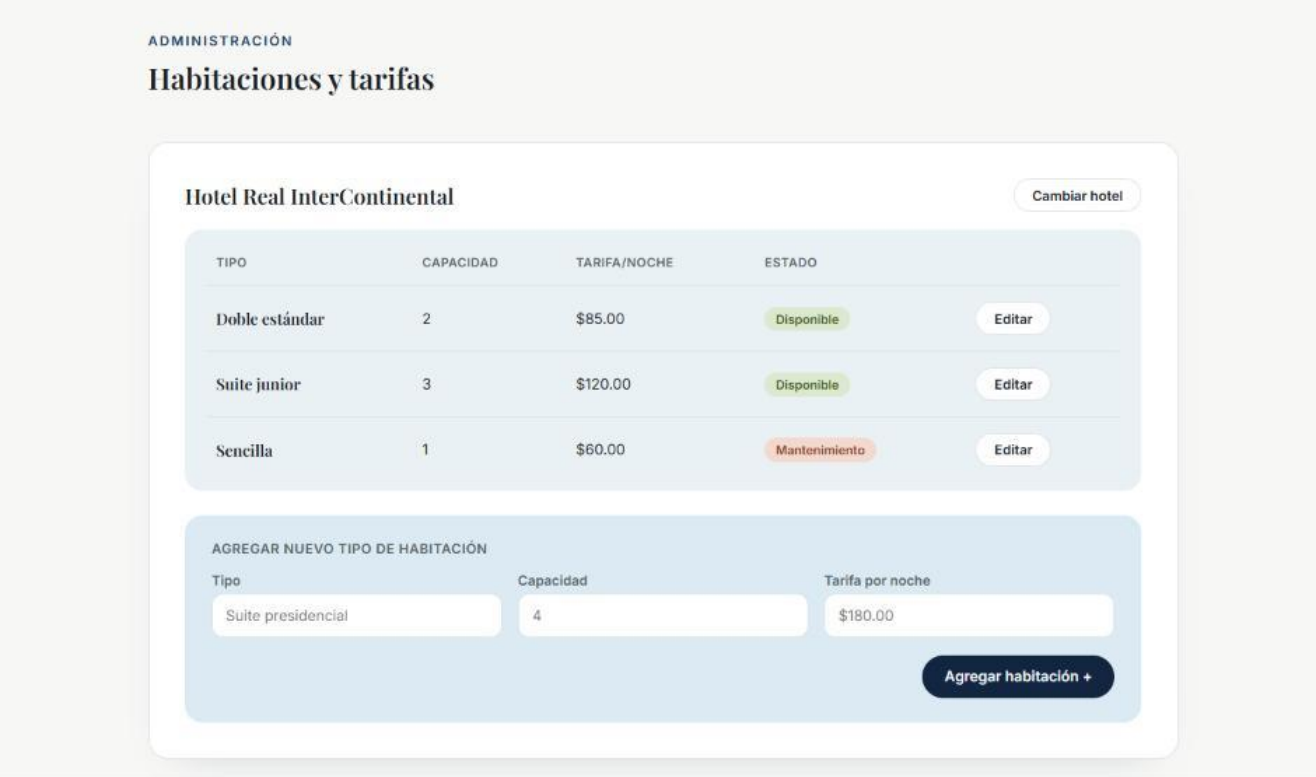
https://trello.com/invite/b/6a9477aad709cbb90177d870/ATTIf83c1649dc9306fb9abdaa2780e8d828F9B5EA45/catedra-dwf

Enlace del repositorio de Git

https://github.com/richialfaro22/Proyecto_Catedra_DWF.git

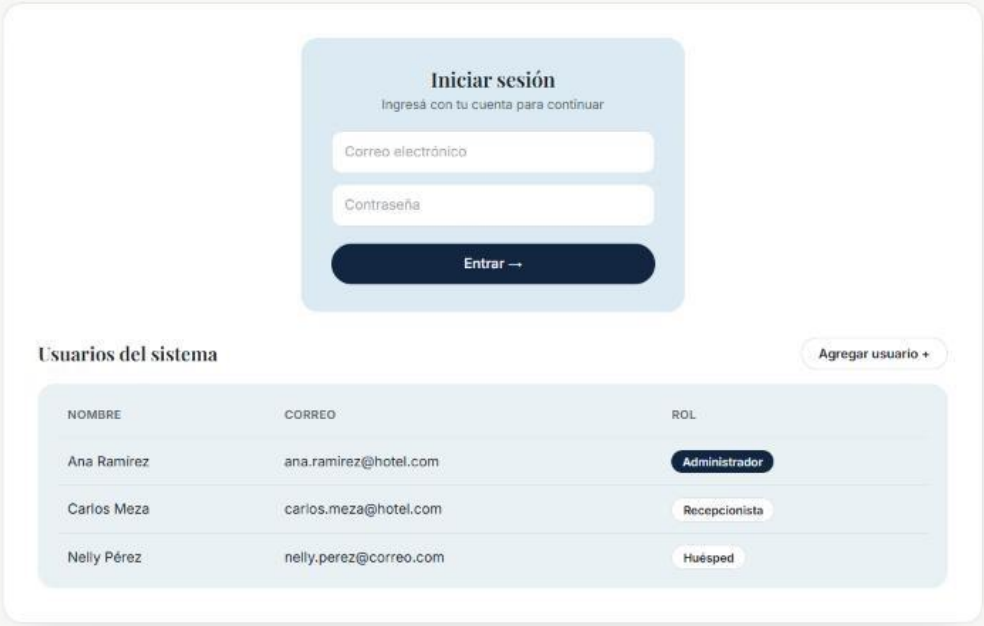
5. Descripción técnica

5.1 Prototipo de la parte FrontEnd



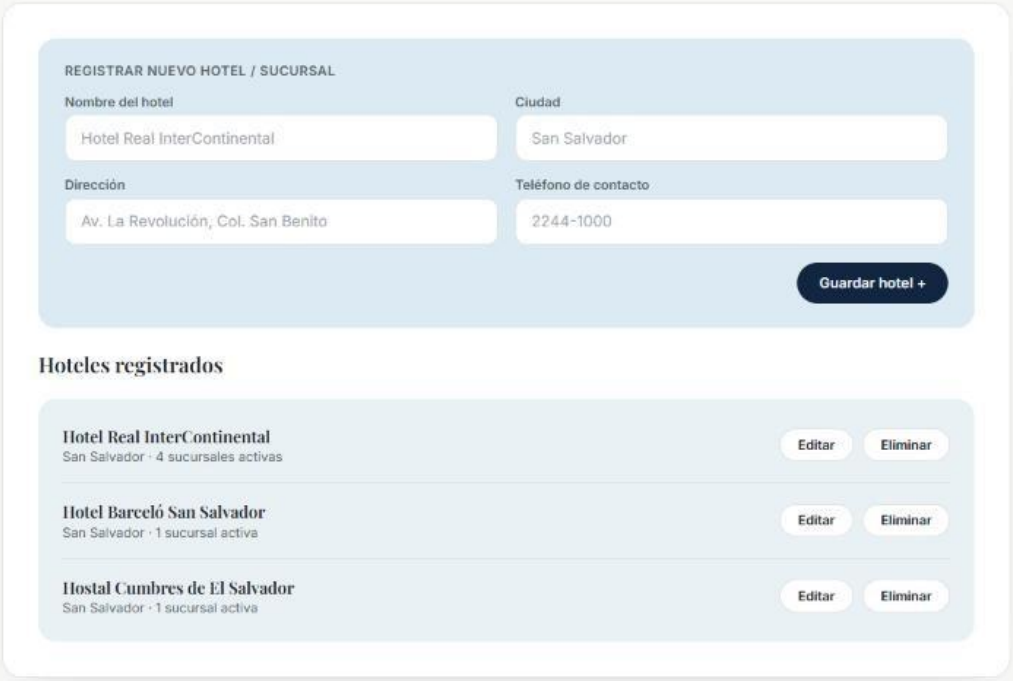
ACCESO AL SISTEMA

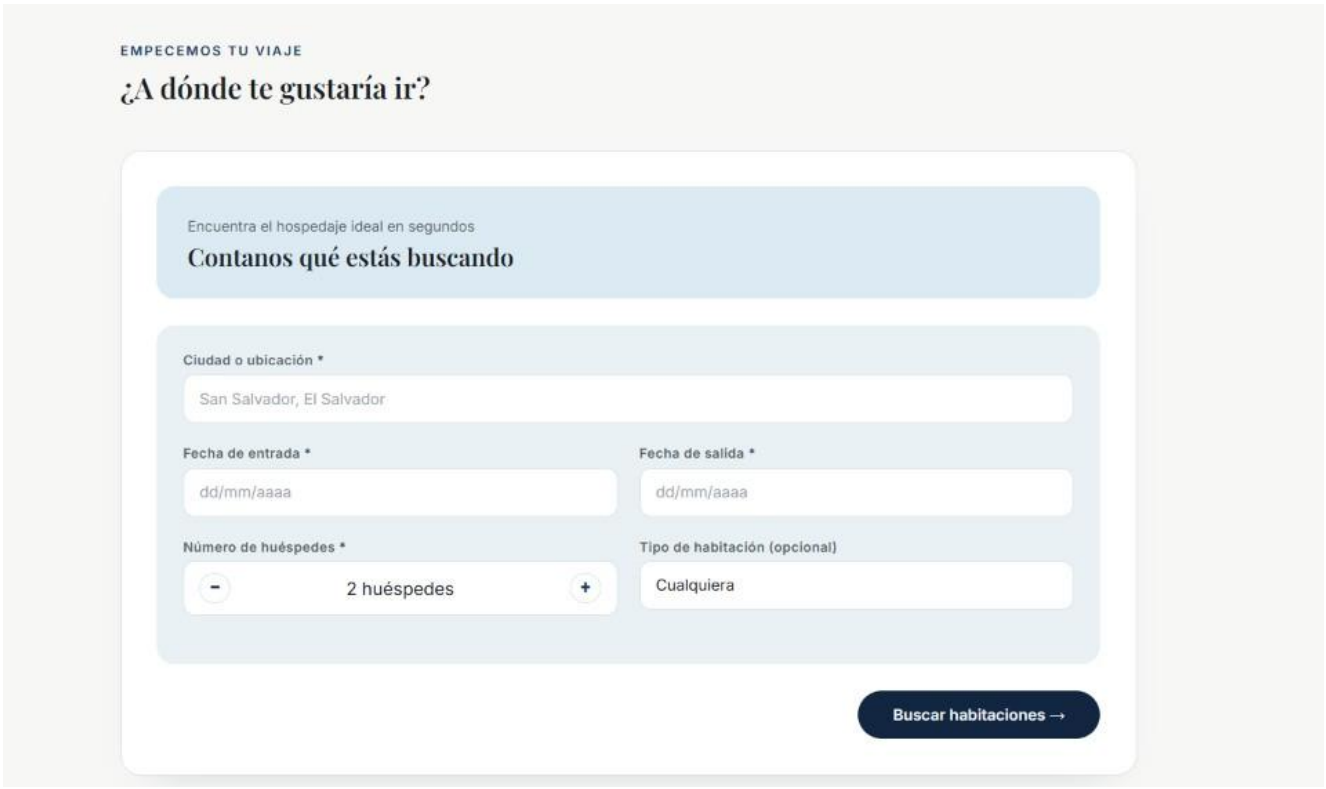
Usuarios y roles

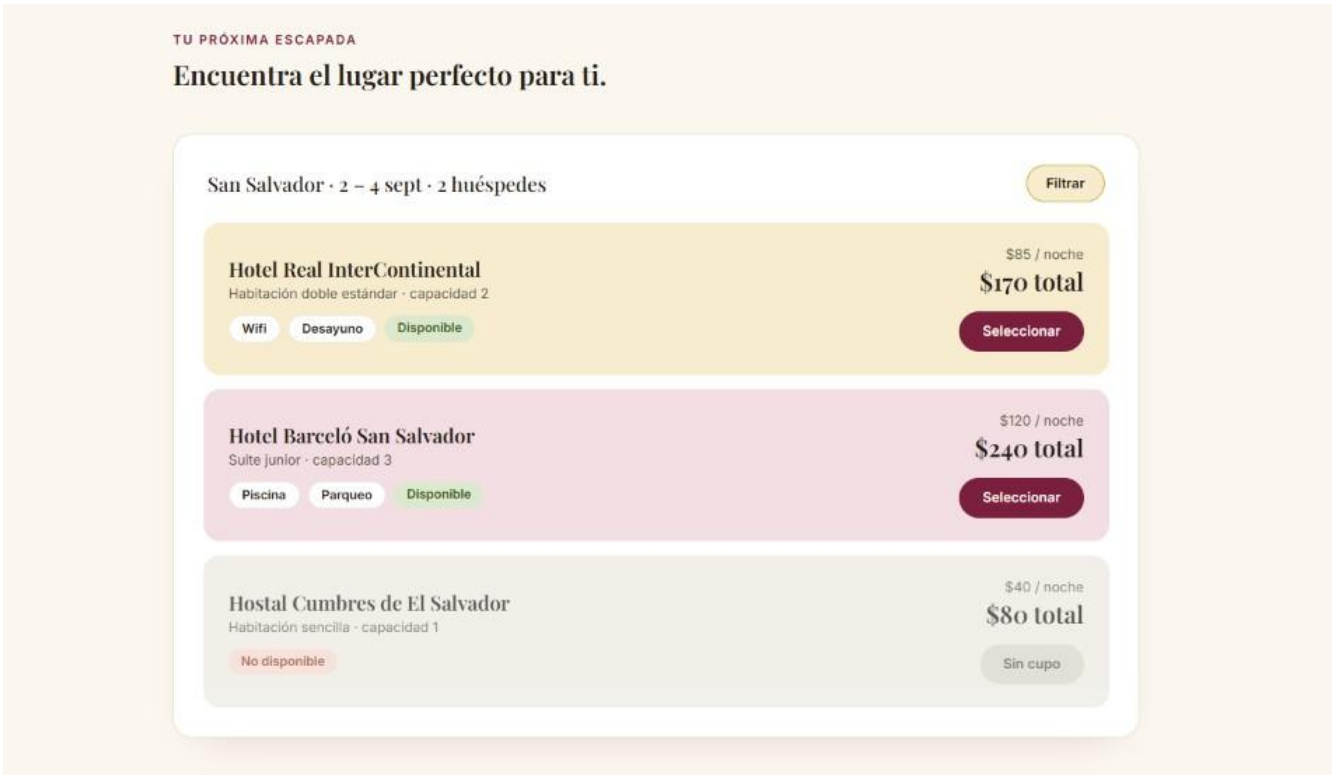


ADMINISTRACIÓN

Tus hoteles y sucursales




EMPECEMOS TU VIAJE
¿A dónde te gustaría ir?
Encuentra el hospedaje ideal en segundos
Contanos qué estás buscando
Ciudad o ubicación *
San Salvador, El Salvador
Fecha de entrada *
dd/mm/aaaa
Fecha de salida *
dd/mm/aaaa
Número de huéspedes *
2 huéspedes
Tipo de habitación (opcional)
Cualquiera
Buscar habitaciones →


TU PRÓXIMA ESCAPADA
Encuentra el lugar perfecto para ti.
San Salvador · 2 – 4 sept · 2 huéspedes
Filtrar
Hotel Real InterContinental
Habitación doble estándar · capacidad 2
Wifi
Desayuno
Disponible
$85 / noche
$170 total
Seleccionar
Hotel Barceló San Salvador
Suite junior · capacidad 3
Piscina
Parqueo
Disponible
$120 / noche
$240 total
Seleccionar
Hostal Cumbres de El Salvador
Habitación sencilla · capacidad 1
No disponible
$40 / noche
$80 total
Sin cupo

YA CASI ES TUYO

Cuéntanos quién se hospeda

Hotel Real InterContinental
Doble estándar · 2 – 4 sept · 2 noches

$170

Nombre completo *
Nicole Méndez

Fecha de nacimiento *
dd/mm/aaaa

Documento de identidad / pasaporte *
04512345-6

Teléfono *
Ej.7000-0000

Correo electrónico *
nombre@correo.com

Solicitudes especiales (opcional)
Ej. cama adicional, piso alto...

Cancelar | Continuar →

ANTES DE CONFIRMAR

Revisa que todo esté correcto

Hotel Real InterContinental
Habitación doble estándar · San Salvador

Huésped	Nelly Pérez
Documento	04512345-6
Fechas	2 – 4 sept 2026
Número de noches	2 noches
Servicios adicionales	Desayuno incluido

Precio total $170.00

Editar datos | Confirmar reserva →

UN PASO MÁS Y ES TUYO

Confirma tu reserva

Monto a pagar

$170.00

Pago pendiente

Método de pago

Tarjeta | Transferencia | Efectivo

Número de tarjeta

•••• •••• •••• 4521

Fecha de pago

30/08/2026

Pagar y confirmar →

¿CAMBIO DE PLANES?

Gestiona tu reserva

Hotel Real InterContinental

2 – 4 sept 2026 · Reserva #A2093

Confirmada

Modificar fechas

Cambia el rango de tu estadía sin perder la habitación

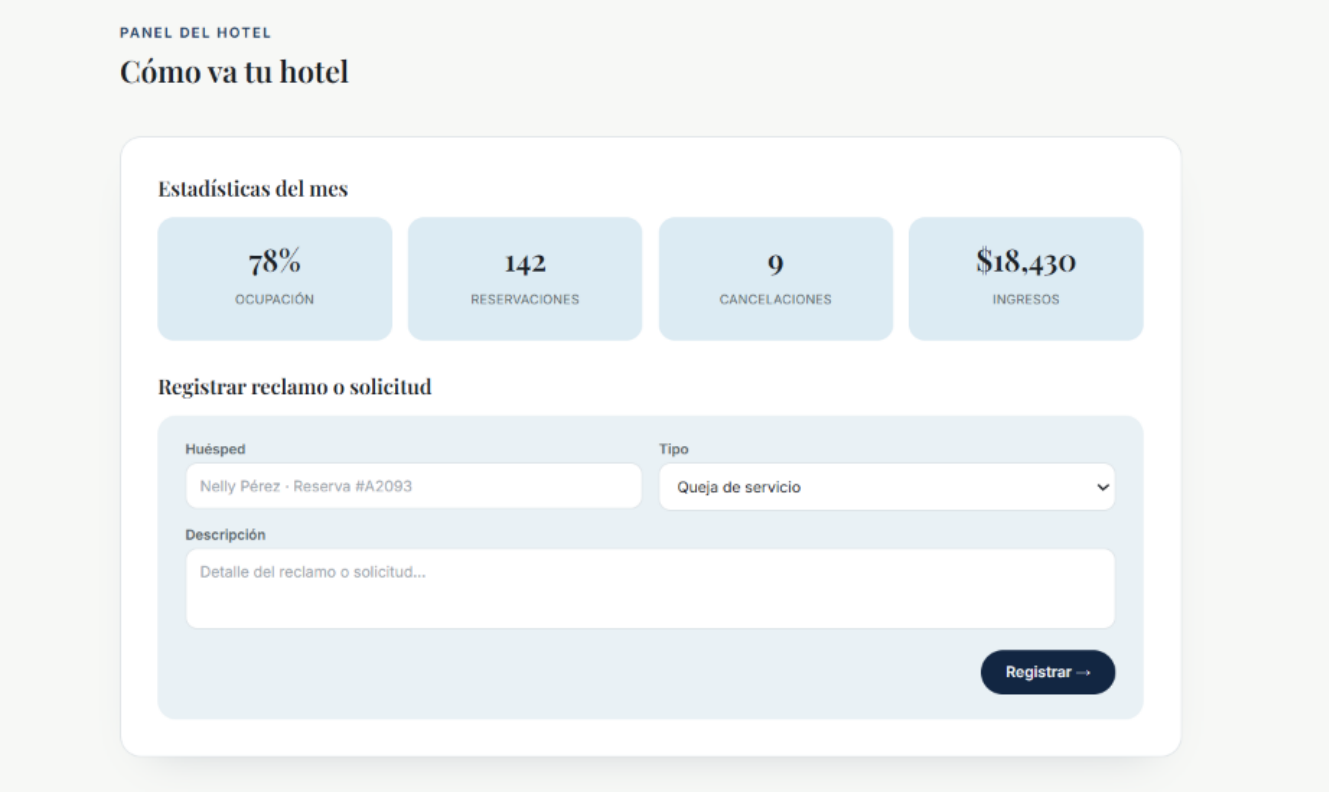
Cancelar reserva

Libera la habitación según la política del hotel

Cancelaciones con menos de 48 horas de anticipación aplican un cargo del 20% sobre el total. El resto se reembolsa al método de pago original.

Volver

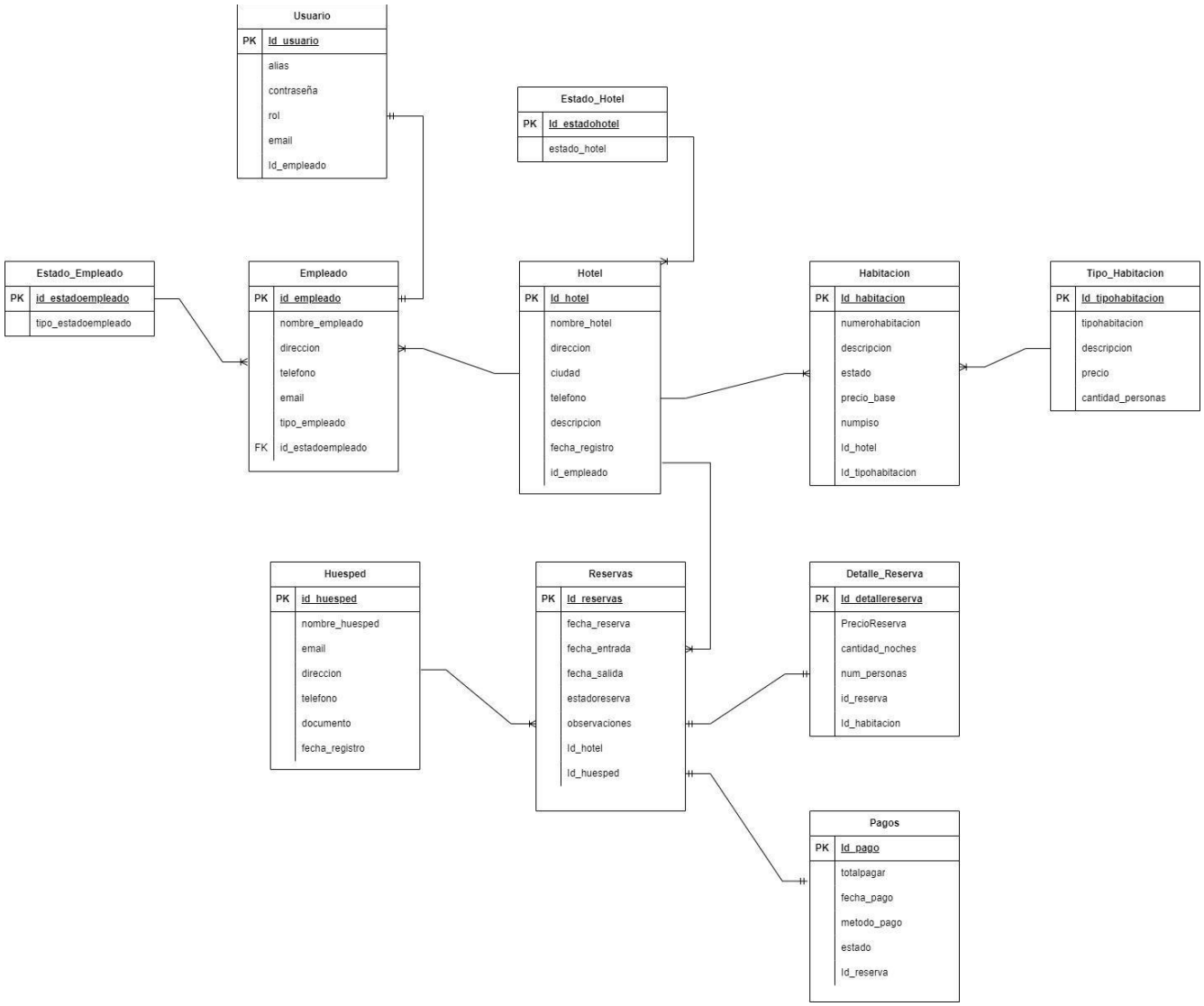
Continuar →



5.2 Gestión de la base de datos

Diagrama entidad-relación

En esta parte se muestra el diseño y la estructura del modelo entidad-relación que se ha planteado.



Diccionario de datos

El presente diccionario de datos describe los campos que forman parte del diagrama mostrado en el punto anterior. Su propósito es dar una idea más clara de cómo será la lógica del desarrollo del proyecto, facilitando la estructura de la base de datos y los tipos de datos que se van a almacenar.

El modelo está orientado a una implementación con una base de datos relacional persistente, como MySQL, y será utilizado posteriormente por el Backend desarrollado con Java y Spring Boot. La estructura contempla información relacionada con usuarios, empleados, hoteles, habitaciones, huéspedes, reservas, detalles de reserva y pagos.

Usuario

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_usuario	INT	Identificador único del usuario del sistema.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
alias	VARCHAR	Nombre o alias utilizado para identificar al usuario.	50	Texto; preferiblemente único	NOT NULL, UNIQUE	—
contraseña	VARCHAR	Contraseña utilizada para autenticación.	255	Cadena de caracteres	NOT NULL; almacenar cifrada/hash	—
rol	VARCHAR	Define el nivel o tipo de acceso del usuario.	30	admin, recepcionista u otros roles definidos	NOT NULL	—
email	VARCHAR	Correo electrónico asociado a la cuenta.	100	Formato de correo válido	NOT NULL, UNIQUE	—
Id_empleado	INT	Identifica al empleado relacionado con la cuenta de usuario.	11	Entero positivo	FK	Empleado.id_empleado

Estado_Hotel

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_estadohotel	INT	Identificador del estado del hotel.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
estado_hotel	VARCHAR	Nombre del estado en el que puede encontrarse un hotel.	30	Activo, inactivo, mantenimiento, etc.	NOT NULL	Hotel.id_estadohotel (recomendado)

Estado_Empleado

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_estadoempleado	INT	Identificador del estado del empleado.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
tipo_estadoempleado	VARCHAR	Descripción del estado laboral o disponibilidad del empleado.	30	Activo, inactivo, suspendido, etc.	NOT NULL	Empleado.id_estadoempleado

Empleado

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_empleado	INT	Identificador único del empleado.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
nombre_empleado	VARCHAR	Nombre completo del empleado.	150	Texto	NOT NULL	—
direccion	VARCHAR	Dirección registrada del empleado.	255	Texto	Puede permitir NULL	—
telefono	VARCHAR	Número telefónico de contacto.	20	Dígitos y símbolos permitidos por formato	Puede permitir NULL	—
email	VARCHAR	Correo electrónico del empleado.	100	Formato de correo válido	UNIQUE recomendado	—
tipo_empleado	VARCHAR	Tipo o cargo desempeñado por el empleado.	50	Recepcionista, administrador, gerente, etc.	NOT NULL	—
id_estadoempleado	INT	Estado actual del empleado.	11	Entero positivo	FK, NOT NULL	Estado_Empleado.id_estadoempleado

Hotel

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones

id_hotel	INT	Identificador único del hotel.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
nombre_hotel	VARCHAR	Nombre comercial del hotel.	150	Texto	NOT NULL	—
direccion	VARCHAR	Dirección física del hotel.	255	Texto	NOT NULL	—
ciudad	VARCHAR	Ciudad o ubicación principal del hotel.	100	Texto	NOT NULL	—
telefono	VARCHAR	Número telefónico de contacto del hotel.	20	Dígitos y símbolos permitidos	Puede permitir NULL	—
descripcion	TEXT	Información adicional sobre el hotel.	Variable	Texto libre	Puede permitir NULL	—
fecha_registro	DATETIME	Fecha y hora en que se registra el hotel.	—	Fecha y hora válida	NOT NULL	—
id_empleado	INT	Empleado relacionado con la administración o registro del hotel.	11	Entero positivo	FK	Empleado.id_empleado
id_estadohotel*	INT	Estado actual del hotel; necesario para materializar la relación con Estado_Hotel del diagrama.	11	Entero positivo	FK recomendado, NOT NULL	Estado_Hotel.id_estadohotel

Habitacion

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_habitacion	INT	Identificador único de la habitación.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
numerohabitacion	VARCHAR	Número o código asignado a la habitación.	10	Texto/número de habitación	NOT NULL, UNIQUE por hotel recomendado	—
descripcion	TEXT	Descripción o características particulares de la habitación.	Variable	Texto libre	Puede permitir NULL	—
estado	VARCHAR	Estado operativo o de disponibilidad de la habitación.	30	Disponible, ocupada, mantenimiento, etc.	NOT NULL	—
precio_base	DECIMAL	Precio base asociado a la habitación.	10,2	Valor mayor o igual a 0	NOT NULL	—

numpiso	INT	Número de piso donde se encuentra la habitación.	11	Entero mayor o igual a 0	NOT NULL	—
Id_hotel	INT	Identifica el hotel al que pertenece la habitación.	11	Entero positivo	FK, NOT NULL	Hotel.id_hotel
Id_tipohabitacion	INT	Identifica el tipo de habitación.	11	Entero positivo	FK, NOT NULL	Tipo_Habitacion.id_tipohabitacion

Tipo_Habitacion

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_tipohabitacion	INT	Identificador único del tipo de habitación.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
tipohabitacion	VARCHAR	Nombre del tipo de habitación.	50	Individual, doble, suite, etc.	NOT NULL	—
descripcion	TEXT	Descripción de las características del tipo de habitación.	Variable	Texto libre	Puede permitir NULL	—
precio	DECIMAL	Precio base definido para el tipo de habitación.	10,2	Valor mayor o igual a 0	NOT NULL	Habitacion.Id_tipohabitacion
cantidad_personas	INT	Cantidad máxima de huéspedes permitida para el tipo.	11	Entero mayor que 0	NOT NULL, CHECK recomendado	—

Huesped

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_huesped	INT	Identificador único del huésped.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
nombre_huesped	VARCHAR	Nombre completo del huésped.	150	Texto	NOT NULL	—
email	VARCHAR	Correo electrónico del huésped.	100	Formato de correo válido	Puede permitir NULL	—
direccion	VARCHAR	Dirección registrada del huésped.	255	Texto	Puede permitir NULL	—
telefono	VARCHAR	Número telefónico del huésped.	20	Dígitos y símbolos permitidos	Puede permitir NULL	—
documento	VARCHAR	Documento de identidad o pasaporte del huésped.	30	Texto/número	NOT NULL, UNIQUE recomendado	—
fecha_registro	DATETIME	Fecha y hora en que se registra el huésped.	—	Fecha y hora válida	NOT NULL	—

Reservas

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
UniqueID / id_reserva	INT	Identificador único de la reserva.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	Detalle_Reserva.id_reserva ; Pagos.id_reserva
fecha_reserva	DATETIME	Fecha y hora en que se crea la reserva.	—	Fecha y hora válida	NOT NULL	—
fecha_entrada	DATE	Fecha prevista de ingreso del huésped.	—	Fecha válida	NOT NULL; posterior a fecha_reserva según regla de negocio	—
fecha_salida	DATE	Fecha prevista de salida del huésped.	—	Fecha válida; mayor que fecha_entrada	NOT NULL	—
estadoreserva	VARCHAR	Estado actual de la reserva.	30	Pendiente, confirmada, cancelada, completada, etc.	NOT NULL	—
observaciones	TEXT	Solicitudes especiales u observaciones asociadas a la reserva.	Variable	Texto libre	Puede permitir NULL	—
Id_hotel	INT	Identifica el hotel donde se realiza la reserva.	11	Entero positivo	FK, NOT NULL	Hotel.id_hotel
Id_huesped	INT	Identifica al huésped que realiza la reserva.	11	Entero positivo	FK, NOT NULL	Huesped.id_huesped

Detalle_Reserva

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
Id_detallereserva	INT	Identificador único del detalle de la reserva.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
PrecioReserva	DECIMAL	Precio aplicado al detalle de la reserva.	10,2	Valor mayor o igual a 0	NOT NULL	—
cantidad_noches	INT	Cantidad de noches correspondientes a la estancia.	11	Entero mayor que 0	NOT NULL, CHECK recomendado	—
num_personas	INT	Cantidad de personas asociadas al detalle de reserva.	11	Entero mayor que 0	NOT NULL, CHECK recomendado	—
id_reserva	INT	Identifica la reserva a la que pertenece el detalle.	11	Entero positivo	FK, NOT NULL	Reservas.id_reserva

Id_habitacion	INT	Identifica la habitación reservada.	11	Entero positivo	FK, NOT NULL	Habitacion.id_habitacion

Pagos

Nombre del campo	Tipo de datos	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
Id_pago	INT	Identificador único del pago.	11	Entero positivo	PK, NOT NULL, AUTO_INCREMENT	—
totalpagar	DECIMAL	Monto total registrado en el pago.	10,2	Valor mayor o igual a 0	NOT NULL	—
fecha_pago	DATETIME	Fecha y hora en que se registra el pago.	—	Fecha y hora válida	NOT NULL	—
metodo_pago	VARCHAR	Método utilizado para realizar el pago.	50	Tarjeta, transferencia, efectivo, etc.	NOT NULL	—
estado	VARCHAR	Estado del pago.	20	Pendiente, aprobado, rechazado, reembolsado, etc.	NOT NULL	—
Id_reserva	INT	Identifica la reserva asociada al pago.	11	Entero positivo	FK, NOT NULL	Reservas.id_reserva

6. Resultados y conclusiones

Como resultado del desarrollo del proyecto se plantea obtener un sistema que permita administrar de una manera más organizada los principales procesos relacionados con las reservas de un hotel. La aplicación estará dividida en un Backend desarrollado con Java y Spring Boot y un Frontend desarrollado con Vue, los cuales se comunicarán mediante una API REST. Esta separación permitirá que cada parte del sistema tenga sus propias responsabilidades. Uno de los puntos principales del proyecto será el proceso de reserva de habitaciones. El sistema deberá permitir realizar una búsqueda utilizando datos como la ubicación, fechas de entrada y salida y cantidad de huéspedes, para posteriormente mostrar las habitaciones que cumplan con las condiciones solicitadas. A partir de esta búsqueda se podrá seleccionar una habitación, registrar los datos del huésped, generar la reserva y registrar la información relacionada con el pago.

También se contará con una base de datos relacional en MySQL, donde se almacenará la información necesaria para el funcionamiento del sistema. El modelo contempla entidades como hoteles, habitaciones, tipos de habitación, huéspedes, reservas y pagos, manteniendo relaciones entre ellas mediante claves primarias y foráneas. Esto permitirá mantener una mayor consistencia de la información y evitar que los datos se encuentren separados o duplicados sin control.

Para concluir, durante la planificación y el desarrollo del proyecto se pudo observar que uno de los puntos más importantes no es solamente programar las funcionalidades, sino definir primero cómo se va a organizar el sistema. La creación del modelo de base de datos, la separación del Backend y el Frontend y la definición de los roles permitieron tener una idea más clara de cómo se desarrollará cada parte del proyecto. Así, se comprende que el desarrollo de este tipo de proyecto no consiste únicamente en escribir código, sino también en planificar, diseñar la estructura de los datos, establecer responsabilidades, realizar pruebas y mantener una buena comunicación.

7. Anexos y documentación de apoyo

Documentación oficial de Spring Boot: https://docs.spring.io/spring-boot/index.html

Guía oficial de Spring para construir servicios web REST: https://spring.io/guides/gs/rest-service

Documentación oficial de Java SE 17: https://docs.oracle.com/en/java/javase/17/

GitHub (control de versiones): https://github.com/

Documentación oficial de MySQL: https://www.mysql.com/

Documentación oficial de PostgreSQL: https://www.postgresql.org/